S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 11

Postlab (OPEN ENDED PRACTICAL)

Aim: To study and implement the **Round Robin (RR) CPU Scheduling Algorithm** in **Operating System** and to calculate the **waiting time** and **turnaround** time of processes using a given time quantum.

Name: Shruti A. Bhabhda

USN: CM24046

Semester/Year: 4th/2nd

Academic session: 2025-2026

Date of Performance:

Date of Submission:

- **Aim:** To study and implement the **Round Robin (RR) CPU Scheduling Algorithm** in **Operating System** and to calculate the **waiting time** and **turnaround time** of processes using a given time quantum.

❖ Objectives:

- To understand the working of the Round Robin (RR) CPU Scheduling Algorithm in Operating System.
- To implement Round Robin scheduling using a fixed time quantum.
- To calculate waiting time and turnaround time for each process.
- To evaluate the performance of the algorithm based on average waiting and turnaround time.
- To analyze how different time quantum values affect CPU scheduling.

❖ Requirements:

1. Hardware Requirements:
   - Computer/Laptop

2. Software Requirements:
   - C Compiler (GCC / Turbo C / CodeBlocks)
   - Operating System: Windows/Linux

❖ Theory:

CPU scheduling is a fundamental function of an **Operating System** that determines which process gets CPU time.

Round Robin Scheduling Algorithm is a **preemptive scheduling algorithm** designed especially for **time-sharing systems.**

In this method:

- Each process is assigned a fixed time slice called **Time Quantum**
- Processes are executed in a **circular queue (FIFO manner)**
- If a process does not complete within its time quantum, it is **preempted** and moved to the end of the queue
- The CPU continues switching between processes until all are completed

This ensures:

- Fairness among processes
- No starvation
- Better response time compared to FCFS

However, performance depends heavily on the size of time quantum:

- **Small quantum** → More context switching (overhead increases)
- **Large quantum** → Behaves like FCFS

➢ **Important Terms**

- **Process:** Program in execution
- **Burst Time (BT):** Time required by process for execution
- **Time Quantum (TQ):** Fixed time assigned to each process
- **Waiting Time (WT):** Time spent in ready queue
- **Turnaround Time (TAT):** Total time from arrival to completion
- **Completion Time (CT):** Time at which process finishes
- **Context Switching**: Switching CPU from one process to another
- **Ready Queue:** Queue of processes waiting for CPU

➢ **Algorithm**

1. Input number of processes (n)
2. Enter burst time for each process
3. Enter time quantum
4. Initialize remaining burst time = burst time
5. Repeat until all processes complete:
    - For each process:
        - If remaining time > TQ
          → Execute for TQ
          → Reduce remaining time
        - Else
          → Execute completely
          → Calculate completion time
6. Calculate:
    - WT = TAT - BT
    - TAT = Completion Time
7. Calculate averages

❖ **Code:**

```
#include <stdio.h>
int main() {
    int n, tq, i, time = 0, remain;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], rt[n], wt[n], tat[n];
    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++) {
    printf("P%d: ", i+1);
    scanf("%d", &bt[i]);
    rt[i] = bt[i];
 }
 printf("Enter Time Quantum: ");
 scanf("%d", &tq);
    remain = n;
    while(remain != 0) {
    for(i = 0; i < n; i++) {
            if(rt[i] > 0) {
            if(rt[i] > tq) {
        time += tq;
        rt[i] -= tq;
    } else {
        time += rt[i];
        wt[i] = time - bt[i];
        rt[i] = 0;
        remain--;
    }
  }
 }
}

for(i = 0; i < n; i++) {
   tat[i] = bt[i] + wt[i];
}

float avg_wt = 0, avg_tat = 0;
 printf("\nProcess\tBT\tWT\tTAT\n");
```

```
for(i = 0; i < n; i++) {
   printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
   avg_wt += wt[i];
   avg_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f", avg_tat/n);

return 0;
 }
```

❖ **Output:**

```
Enter number of processes: 4
Enter Burst Time:
P1: 15
P2: 10
P3: 8
P4: 5
Enter Time Quantum: 3

Process BT  WT  TAT
P1  15  23  38
P2  10  25  35
P3  8   23  31
P4  5   18  23

Average Waiting Time = 22.25
Average Turnaround Time = 31.75

=== Code Execution Successful ===
```

❖ Conclusion:

The **Round Robin CPU scheduling algorithm** was successfully implemented in **Operating System**. The waiting time and turnaround time for all processes were calculated using a fixed time quantum. The algorithm ensures fair CPU allocation to all processes and prevents starvation. It was observed that the performance of the algorithm depends on the size of the time quantum, as smaller values increase context switching while larger values reduce responsiveness.

❖ Discussion Question:

1.What is Round Robin scheduling?

2. What is the role of time quantum?

3. Why is Round Robin considered fair?

4. What happens if time quantum is too small?

5. What happens if time quantum is too large?

❖ References:

https://www.geeksforgeeks.org/
https://www.tutorialspoint.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26